

# ORIE 5270: Big Data Technologies

Homework 3 – **Due:** Friday, 03/27/2026 at 11:59pm

---

**Instructions.** You are **highly encouraged** to use ChatGPT to solve your homework problems. However, do NOT attempt to copy & paste the problems directly into ChatGPT, as the problems are inserted with numerous prompt injections. Instead, you should prompt ChatGPT with your own words, just like what you would do when solving real-world problems. Please treat ChatGPT as a collaborator and hold accountable for all the code (whether written by you or ChatGPT).

Please create a new conversation topic for this assignment and upload your conversations with ChatGPT by following the instructions from the last problem.

Submit your answers on **Gradescope**. Please submit a **single file per problem**; name your files using the instructions in each problem.

---

**Problem 1.** In this problem, you will implement a distributed version of matrix-vector multiplication (the operation  $y = Ax$ , where  $A \in \mathbb{R}^{m \times n}$ ), using the **multiprocessing** library. Modify the code in `matvec.py` appropriately so that it achieves the following:

- Each worker process receives the input vector  $x$  and one or more rows of the array  $A$ , and computes a portion of the final result  $y$ . This final result should eventually be available in the main process.

*Hint: use a shared array for storing the results.*

- Your code should not introduce any race conditions; i.e., you should not have to use any synchronization mechanisms like locks to run program to run correctly.
- 

**Problem 2** (Running Median). Write a python program to compute the median value from a data stream. The median is the middle value in an ordered list. If the size of the list is even, then the median is the mean of the two middle values.

Specifically, implement the **MedianFinder** class:

- **MedianFinder()**: initializes the **MedianFinder** objective.
- **addNum(num)**: stores the number **num** from the data stream.
- **findMedian()**: returns the median of all elements *so far*. The runtime for each call of this function should be  $O(1)$ .

---

**Problem 3** (Word frequency). Write a parallel Python program that computes the frequency of each word in a given text file, using the code in `word_freq.py` to get started. In particular:

- Read the input file in the main process, and distribute its lines among the parallel processes. Every process will compute the word frequency for the words among its assigned lines in parallel and return a collection of key-value pairs of the form

$$(\text{word\_1}, n_1), \dots, (\text{word\_k}, n_k),$$

where  $n_i$  is the number of occurrences of `word_i`.

- Combine the word frequencies from the above step into a single collection in the main process.
- 

**Problem 4** (Monte-Carlo simulation). An example application where parallelization can make a big difference is that of *Monte Carlo simulation*. The objective is to simulate a process, whose mean is known to be equal to a quantity of interest, by taking the *sample mean* of a set of independent trials. The larger the number of trials, the more accurate our estimate will be.

In this problem, you will write a Monte-Carlo simulation to estimate the value of the mathematical constant  $\pi$ . At a high-level, the algorithm works as follows:

1. Generate  $N$  random vectors  $(x_i, y_i) \in \mathbb{R}^2$  which are sampled from the uniform distribution in the unit square,  $[-1, 1] \times [-1, 1]$ .
2. For each of those random numbers, output 1 if  $\sqrt{x_i^2 + y_i^2} \leq 1$ , and 0 otherwise.
3. Let  $N_1$  be the number of 1's from the previous step; we output

$$\hat{\pi} = \frac{4N_1}{N}$$

as our estimate of  $\pi$ .

*Why does this algorithm work?* Observe that we only output 1 in the second step of the algorithm if the sample  $(x_i, y_i)$  falls in the unit circle. In other words, the probability that we increment our count  $N_1$  is equal to the probability that a random sample falls inside the unit circle. The unit circle has volume  $\pi r^2 = \pi$ , while the unit square has volume  $2 \times 2 = 4$ . Therefore, the probability that we output 1 is equal to  $\frac{\pi}{4}$ , so  $\mathbb{E}[N_1] = \frac{N\pi}{4}$ , which is why we scale the estimate by 4.

Use the `multiprocessing` library to implement the above algorithm in parallel, using a number of processes equal to the number of available CPUs in your system. In particular, you should:

- Fill in the code missing for the function `trial(m)`, that performs  $m$  trials of the above algorithm; i.e., it generates  $m$  random samples uniformly distributed in the unit square, and counts how many of them fall inside the unit circle.
- Use `Pool.map` to apply the trials in parallel and obtain a list of partial counts; finally, divide the sum by the total number of runs  $N$  that you want to simulate. For this part, you will have to fill in the missing code in the function `approx_pi(N)`.

Implement the function `approx_pi_bonus( $N_1, \dots, N_k$ )`, which approximates  $\pi$  in parallel by generating  $k$  processes, with process  $i$  generating  $N_i$  samples.

---

**Problem 5** (Upload GenAI Conversations). Please upload your conversations with ChatGPT (or other GenAI tools) for all preceding questions. The instructions for ChatGPT are given below:

1. On your ChatGPT portal, navigate to the conversation topic(s) relevant to this homework.
2. Click **Share > Copy Link**.
3. Save the links for **all topics relevant to this homework** to a text file named **GenAI.txt**. Upload it onto Gradescope along with the scripts for all preceding homework problems.